Bliss India- A Full Stack E-Commerce Website

Summer Training Report submitted in partial fulfilment of the requirement for the degree of

Bachelor of Technology

in

Computer Science & Engineering

By

Harsh Jain (11915002721)

To

Computer Science and Engineering Department

Maharaja Surajmal Institute of Technology

Affiliated to Guru Gobind Singh Indraprastha University

Janakpuri, New Delhi-58

CERTIFICATE

ACKNOWLEDGEMENT

The success of every summer training, regardless of its scale, owes much to the efforts of numerous wonderful individuals who have consistently offered valuable advice and a helping hand. I express sincere appreciation for the inspiration, support, and guidance provided by those instrumental in the success of this project.

I, Harsh Jain, a student of Maharaja Surajmal Institute of Technology, New Delhi, pursuing Computer Science & Engineering, extend heartfelt gratitude to Coloration Medicare LLP for their confidence in me. I am particularly thankful to Mr. Pramod Kumar Jha for overseeing my 'Web Development Internship Program.' I feel deeply honored to acknowledge Mr. Pramod for timely resource provision and valuable insights that significantly contributed to the successful completion of my training.

Special thanks go to my mentor, Mr. Pramod Kumar Jha, Director of Coloration Medicare LLP, whose assistance was crucial in bringing the project to fruition. I also express my gratitude to the faculty members of the Computer Science & Engineering department at Maharaja Surajmal Institute of Technology, New Delhi, for their critical advice and guidance, which played an integral role in the realization of this training.

Lastly, but certainly not least, I extend profound gratitude to my family members and friends, who have been a constant source of inspiration throughout the preparation of this training report.

ABSTRACT

BlissIndia is a jewelry brand dedicated to showcasing an extensive and sophisticated collection, tailored for customers with refined tastes who value elegance. By incorporating cutting-edge technology, the platform seamlessly blends innovation to provide a secure and user-friendly shopping experience. Our thoughtfully curated selection includes a diverse range of styles and materials, featuring intricately crafted rings and timeless necklaces. Each piece is a unique expression of artistic craftsmanship, ensuring that our jewelry stands out for its individuality and attention to detail.

Our website is dedicated to streamlining the shopping experience, allowing customers to order directly and enjoy additional discounts. With features such as wishlist and cart options, users can curate their selections conveniently. The search functionality simplifies product discovery, and the ability to leave reviews fosters customer engagement and informed decision-making. To enhance transparency, customers can download receipts for their orders and easily check their order history. This feature enables users to track and review their past purchases. Our commitment to customer satisfaction is further emphasized by a responsive support system and secure payment gateways, ensuring a safe and reliable transaction environment. Overall, our website is tailored to prioritize user convenience, making every step of the shopping journey efficient and enjoyable.

TABLE OF CONTENTS

CONTENT	PAGE NO
Certificate	I
Acknowledgement	II
Abstract	III
Table Of Contents	IV
CHAPTER	**PAGE NO**
1. INTRODUCTION	
1.1. Need & Objective	1
1.2. Methodology	2
1.3. Software Used	3
1.4. About Organisation	17
2. PROJECT DESIGN	
2.1. Client Side	18
2.2. Server Side	22
3. IMPLEMENTATION	
3.1 Client Side Implementation	24
3.2 Server Side Implementation	29
4. RESULT AND DISCUSSION	
4.1 Result	33
4.2 Discussion	34
5. FUTURE SCOPE AND CONCLUSION	
5.1 Future Scope	35
5.2 Conclusion	36
References	38

1. INTRODUCTION

1.1 NEED AND OBJECTIVE

The necessity and goal of embarking on the BlissIndia project are rooted in the imperative need for the company's growth and national outreach. As the primary objective, the development of a company website serves as a strategic initiative to propel Coloration Medicare LLP into broader geographical domains across the country. Recognizing the pivotal role of an online presence in today's business landscape, establishing a comprehensive and user-friendly website emerges as a crucial step towards enhancing the company's visibility, accessibility, and overall market influence.

In essence, the objective of the BlissIndia project is to leverage the potential of a well-crafted website to effectively showcase the company's offerings, services, and values. The website becomes a virtual gateway, not only serving as an informative platform but also as a dynamic tool for engaging with a wider audience. This aligns with the broader objective of fostering company growth, reinforcing its brand presence, and ensuring a meaningful connection with stakeholders nationwide. The strategic importance of a company website in today's digital age underscores the significance of this project in the context of advancing the company's reach and impact.

The key goals of my summer internship at Coloration Medicare LLP included:

1. Gaining practical experience in web development and mastering the MERN (MongoDB, Express.js, Node.js) stack alongside ReactJS.

2. Developing a comprehensive understanding of both frontend and backend development within a client-server database model.

3. Successfully completing a hands-on project, specifically an e-commerce website (BlissIndia), to validate and apply the acquired knowledge and skills.

4. Strengthening my problem-solving and project management capabilities by tackling real-world challenges in web development.

These objectives not only directed the course of my training but also provided constant motivation and purpose throughout the project. They served as a roadmap, helping me focus on the essential aspects required to achieve my goals.

1.2 METHODOLOGY

1.2.1 Project Initiation and Feature Identification:

The inception of our website development journey involved an extensive and meticulous brainstorming session. During this phase, we conscientiously compiled a comprehensive list of all the desired features, envisioning the functionality and user experience we aimed to deliver.

1.2.2 Adoption of MVC Architecture:

To instil structure and organization into our implementation process, we strategically embraced the Model-View-Controller (MVC) architectural pattern. This decision provided a well-defined framework for managing data, user interface, and application logic, laying a robust foundation for the development endeavour.

1.2.3 Dynamic Development Approach:

As the development journey unfolded, we confronted novel challenges and complexities inherent in crafting a sophisticated website. In response, our approach remained dynamic and adaptive. Rather than succumbing to roadblocks, we proactively sought solutions through continuous research and innovation.

1.2.4 Iterative Feature Implementation:

Each feature identified in our initial brainstorming session underwent iterative implementation. This iterative methodology allowed us to build and refine features incrementally, ensuring that our development process was not only responsive but also capable of addressing emerging requirements in real-time.

1.2.5 Flexibility to Evolving Project Needs:

The adaptive nature of our methodology empowered us to remain flexible and responsive to the evolving needs of the project. Whether it was user feedback, technological advancements, or changing project requirements, our development

approach was agile, ensuring that the website aligned seamlessly with the project's dynamic landscape.

1.2.6 Fusion of Planning and Execution:

The successful navigation of the intricacies of development was a result of a harmonious fusion of thoughtful planning and meticulous execution. Our commitment to adhering to the MVC structure provided a roadmap, while our problem-solving mindset ensured that obstacles were not impediments but rather opportunities for improvement.

1.2.7 Robust and Feature-Rich Outcome:

This comprehensive and adaptive development journey culminated in the creation of a robust and feature-rich website. The fusion of strategic planning, architectural soundness, and an agile development approach collectively contributed to the realization of a digital platform that not only met but exceeded the envisioned goals and expectations.

1.3 SOFTWARE USED

In the development of the website, a diverse set of tools and technologies were instrumental in shaping the project. The backend, built on Node.js, was enhanced with the integration of SwiperJS for dynamic and responsive carousel functionality. Morgan and Rotating File Stream played key roles in logging and managing server-side activities, ensuring efficient debugging and error tracking. The templating engine EJS facilitated the dynamic rendering of HTML content on the server side, contributing to a seamless user experience. Noty, a notification library, was employed to implement user-friendly alerts and messages. Nodemailer played a crucial role in enabling server-side email functionality, ensuring effective communication with users. For styling and design, SASS (Syntactically Awesome Stylesheets) provided a powerful and structured approach to CSS development. The build process was optimized with Gulp, streamlining tasks such as code compilation and file optimization. This diverse toolset, encompassing SwiperJS, Morgan, Rotating File Stream, EJS, Noty, Nodemailer, SASS, Gulp, and others, collectively contributed to the successful implementation and functionality of the website, enriching both the user interface and backend processes.

1.3.1 HTML

1.3.1.1 Introduction to HTML

HTML, or Hypertext Markup Language, is the fundamental language of the web. It is used to structure content on web pages, defining the layout and presentation of information. HTML consists of a series of elements, each enclosed by tags, that describe different parts of a webpage.

HTML is the backbone of any web page, and it provides the structure that enables web browsers to render content in a human-readable form. It is essential for creating everything from simple text documents to complex multimedia-rich websites. HTML also plays a significant role in ensuring accessibility, as it allows developers to provide semantic meaning to content, making it understandable to both humans and assistive technologies like screen readers.

HTML is designed to be easy to learn and use, making it an ideal starting point for anyone interested in web development. It provides a foundation for adding more advanced features and interactivity to web pages through other technologies like CSS (Cascading Style Sheets) and JavaScript.

1.3.1.2 Example HTML Code and Explanation

```
<!DOCTYPE html>
<html>
    <head>
        <title>My First Web Page</title>
    </head>
    <body>
        <h1>Welcome to My Web Page</h1>
        <p>This is a simple example of HTML content. </p>
    </body>
</html>
```

Basic HTML document starts with an opening <html> tag, inside it there are <head>

</head> tags, after head we have body of the HTML page within <body> tag and at last HTML document is completed with closing </html> tag. Since it is the introduction to HTML structure few points can describe the function of basic HTML tags:

1. <!DOCTYPE html>: This is a document type declaration and specifies that the document is written in HTML5. It tells the browser how to interpret the rest of the code.

2. <html>: The root element of an HTML document. All other elements are nested inside this element. It defines the beginning and end of the HTML document.

3. <head>: This section contains metadata about the document, such as the document's title, character encoding, and links to external resources like stylesheets or scripts.

4. <title>: The title element is placed within the <head> section and defines the title of the web page. The text inside this tag is displayed in the browser's title bar or tab when the page is open.

5. <body>: The body element contains the main content of the web page that will be visible to the user. This is where you place text, images, links, and other elements that make up the actual content of the page.

6. <h1>: This is a heading element. Headings are used to define the structure of a document and provide visual cues to readers. The <h1> tag represents the highest- level heading and is typically used for the main title of the page.

7. <p>: The <p> tag represents a paragraph of text. It is used to structure and format text content. Paragraphs are separated by a line break or some space to make the content more readable.

This is a very basic example, but it illustrates the structure of an HTML document. HTML can become more complex as you add more elements, formatting, and interactivity, but it always follows a similar structure of elements enclosed by tags, single or double.

1.3.2 CSS

1.3.2.1 Introduction to CSS

Cascading Style Sheets, commonly known as CSS, is a fundamental technology used in web development. It plays a crucial role in controlling the visual presentation and layout of web pages. CSS allows web developers to define how HTML elements are displayed, specifying attributes such as colors, fonts, spacing, and positioning. It separates the structure (HTML) and presentation (CSS) of a web page, making it easier to create and maintain consistent, attractive, and responsive designs.

CSS is essential for achieving a cohesive and visually appealing user experience on the web. It offers flexibility and control over the look and feel of web content, making it a vital tool for web designers and developers. With CSS, you can customize the appearance of text, images, backgrounds, borders, and more, ensuring that your web pages not only convey information effectively but also provide an engaging and aesthetically pleasing interface for users.

1.3.2.2 Example CSS Code and Explanation

```
body {
    background-color: #f0f0f0; font-family: Arial, sans-serif; margin: 0;
    padding: 0;
}
h1 {
    color: #333; font-size: 36px;
    text-align: center;
}
div.content p {
    color: #666; font-size: 18px; line-height: 1.5;
}
```

```
a {
    color: #0074d9;
    text-decoration: none;
}
a: hover {
    color: #0056b3;
}
```

1. /* This is a CSS comment */: These are CSS comments that provide explanations and notes but are not rendered in the webpage.
2. body: This selector targets the <body> element of the webpage, setting various styles for the entire page.
3. {}: The curly braces enclose a declaration block that contains the styles to be applied to the selected element.
4. background-color: #f0f0f0;: This property changes the background color of the entire page to a light gray (#f0f0f0).
5. font-family: Arial, sans-serif;: This property sets the default font family to Arial, with a fallback to sans-serif fonts if Arial isn't available on the user's system.
6. margin: 0; and padding: 0;: These properties remove the default margin and padding from the <body> element.
7. h1: This selector targets all <h1> elements on the page.
8. color: #333;: This property changes the text color of <h1> elements to a dark gray (#333).
9. font-size: 36px;: This sets the font size of <h1> elements to 36 pixels.
10. text-align: center;: This centers the text within <h1> elements.
11. div.content p: This selector targets all <p> elements inside a with the class "content".

12. line-height: 1.5;: This adjusts the line height of the <p> elements for better readability.

13. a: This selector targets all links on the page.

14. text-decoration: none;: This removes the underline from links to make them appear as plain text.

15. a:hover: This selector changes the link colour when hovered over by the mouse.

This example illustrates how CSS can be used to style various elements of a webpage, such as the background, fonts, colours, and text alignment. CSS selectors target specific HTML elements, allowing you to apply different styles to different parts of your web page.

1.3.3 JavaScript

1.3.3.1 Introduction to JavaScript

JavaScript is a versatile, high-level, and dynamic programming language that plays a pivotal role in web development. As the core technology behind interactive and dynamic web applications, JavaScript enables developers to bring web pages to life by adding functionality, interactivity, and responsiveness. It is a fundamental part of the modern web ecosystem, working seamlessly alongside HTML and CSS. JavaScript allows developers to create complex applications, control the behaviour of web pages, and interact with users in real-time. It's commonly used for tasks like form validation, handling user input, making asynchronous requests to servers (Ajax), and manipulating the Document Object Model (DOM) to update and change page content without requiring a full page reload. JavaScript's importance extends beyond the web; it is now used in various application domains, including server-side development (Node.js), mobile app development (React Native), and game development (using libraries like Three.js).

1.3.3.2 Example JavaScript Code and Explanation

```
// Define a function that calculates the factorial of a number function calculateFactorial(number) {

    let result = 1;

    for (let i = 1; i <= number; i++) {
```

```
        result *= i;
    }
    return result;
}
// Get user input
const userInput = prompt("Enter a number to calculate its factorial:");
// Convert user input to a number
const inputNumber = parseInt(userInput);
// Calculate the factorial
const factorial = calculateFactorial(inputNumber);
// Display the result
console.log(`The factorial of ${inputNumber} is ${factorial}`);
```

1. function calculateFactorial(number): This is the declaration of a JavaScript function named calculateFactorial. It takes a parameter number and calculates the factorial of that number using a for loop.
2. let result = 1;: A variable result is initialized with a value of 1. This variable will store the result of the factorial calculation.
3. for (let i = 1; i <= number; i++): This is a for loop that iterates from 1 to the given number, inclusive.
4. result *= i;: Inside the loop, the result variable is updated to calculate the factorial.
5. return result;: The function returns the calculated factorial.
6. const userInput = prompt("Enter a number to calculate its factorial:");: This line uses the prompt method to interact with the user and collect their input.
7. const inputNumber = parseInt(userInput);: The user's input is converted from a string to an integer using the parseInt function.

8. const factorial = calculateFactorial(inputNumber);: The calculateFactorial function is called to calculate the factorial of the user's input.

9. console.log(The factorial of ${inputNumber} is ${factorial});: Finally, the result is displayed in the console using console.log.

1.3.4 SwiperJS

1.3.4.1 Introduction to SwiperJS

SwiperJS, often known simply as Swiper, is an open-source JavaScript library developed and maintained by a dedicated community of developers. This influential front-end library is widely recognized for its prowess in creating dynamic and responsive touch-enabled sliders for web applications. Swiper's core objective is to offer a streamlined and effective method for constructing interactive and adaptable carousel components. Employing a declarative approach, Swiper simplifies the process of designing and managing intricate sliders within web interfaces, contributing to a more efficient development and maintenance workflow.

1.3.4.2 Example SwiperJS Code

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <link rel="stylesheet" href="https://unpkg.com/swiper/swiper-bundle.min.css" />

  <style>

    .swiper-container {

      width: 100%;

      height: 100%;

    }

    .swiper-slide {
```

```
      text-align: center;
      font-size: 18px;
      background: #fff;
    }
    img {
      max-width: 100%;
      height: auto;
    }
  </style>
</head>
<body>
  <div class="swiper-container">
    <div class="swiper-wrapper">
      <div class="swiper-slide"><img src="image1.jpg" alt="Slide 1"></div>
      <div class="swiper-slide"><img src="image2.jpg" alt="Slide 2"></div>
      <div class="swiper-slide"><img src="image3.jpg" alt="Slide 3"></div>
    </div>
    <!-- Add Pagination -->
    <div class="swiper-pagination"></div>
    <!-- Add Navigation -->
    <div class="swiper-button-next"></div>
    <div class="swiper-button-prev"></div>
  </div>
  <script src="https://unpkg.com/swiper/swiper-bundle.min.js"></script>
```

```
  <script>
    var swiper = new Swiper('.swiper-container', {
      slidesPerView: 1,
      spaceBetween: 20,
      navigation: {
        nextEl: '.swiper-button-next',
        prevEl: '.swiper-button-prev',
      },
      pagination: {
        el: '.swiper-pagination',
        clickable: true,
      },
    });
  </script>
</body>
</html>
```

1.3.5 EJS

1.3.5.1 Introduction to EJS

EJS, or Embedded JavaScript, is a dynamic templating engine designed to seamlessly integrate JavaScript logic into HTML, enabling the creation of dynamic web content. With its straightforward syntax and compatibility with Express.js, EJS simplifies server-side rendering, making it an ideal choice for building responsive and engaging web applications. Developers can leverage EJS to inject data, implement conditional statements, and incorporate loops, fostering the creation of scalable and maintainable projects. This lightweight and efficient templating engine empowers developers to effortlessly blend functionality with aesthetics, resulting in modern, interactive, and

content-rich websites. EJS stands as a versatile tool, offering a concise and elegant solution for dynamic web development.

1.3.5.2 Example EJS Code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>EJS Example</title>
</head>
<body>
  <h1>Product List</h1>
  <ul>
    <% products.forEach(product => { %>
      <li><%= product.name %> - $<%= product.price.toFixed(2) %></li>
    <% }); %>
  </ul>
  <h2>Add a New Product</h2>
  <form action="/add-product" method="post">
    <label for="name">Product Name:</label>
    <input type="text" id="name" name="name" required>
    <label for="price">Price:</label>
    <input type="number" id="price" name="price" step="0.01" required>
    <button type="submit">Add Product</button>
  </form>
```

```
</body>

</html>
```

1.3.6 NodeJS

1.3.6.1 Introduction to NodeJS

Node.js is a powerful, open-source, server-side runtime environment that allows developers to build scalable and high-performance network applications. It is based on the V8 JavaScript engine developed by Google and utilizes an event-driven, non-blocking I/O model, making it well-suited for building real-time, data-intensive, and networked applications. Node.js is primarily used for building server-side applications, APIs, and web servers but is also employed for various other purposes, including building desktop applications.

1.3.6.2 Example NodeJS Code and Explanation

```
 const http = require('http');

const server = http.createServer((req, res) => {

        res.writeHead(200, { 'Content-Type': 'text/plain' }); res.end('Hello, World!\n');

});

const port = process.env.PORT || 3000; server.listen(port, () => {

        console.log(`Server is running on port ${port}`);

});
```

1. We start by importing the Node.js http module, which provides functionality for creating HTTP servers and handling HTTP requests and responses.
2. We create an HTTP server using the http.createServer method, which takes a callback function that gets executed when an HTTP request is received.
3. Inside the callback function, we set the response's status code to 200 (OK) and the content type to 'text/plain'. Then, we use res.end() to send the "Hello, World!" message to the client.

4. We define a port for the server to listen on. It uses the value of the PORT environment variable if available, or defaults to port 3000.

5. The server.listen method starts the server, and we log a message indicating which port the server is running on.

1.3.7 ExpressJS

1.3.7.1 Introduction to ExpressJS

Express.js, often simply referred to as Express, is a popular open-source web application framework for Node.js. It provides a robust set of features for building web and mobile applications, as well as APIs. Express simplifies the development of server-side applications by offering a minimal and flexible structure, making it an excellent choice for building web servers, RESTful APIs, and full-stack web applications. It is designed to work seamlessly with Node.js and is widely used in the Node.js ecosystem.

1.3.7.2 Example ExpressJS Code and Explanation

```
const express = require('express'); const app = express();

const port = process.env.PORT || 3000;

app.get('/', (req, res) => { res.send('Hello, Express!');});

app.listen(port, () => {

        console.log(`Express server is running on port ${port}`);

});
```

1. We start by importing the Express.js framework with require('express'). This initializes an Express application instance that we store in the app variable.

2. We define a port for the server to listen on, similar to the previous Node.js example. It uses the value of the PORT environment variable if available, or defaults to port 3000.

3. We create a route using app.get('/', ...), where '/' is the route path. When a GET request is made to the root path ('/'), the callback function is executed.

4. Inside the callback function, we use res.send() to send the "Hello, Express!" message as the HTTP response.

5. The app.listen method starts the Express server, and we log a message indicating which port the server is running on.

1.3.8 MongoDB

1.3.8.1 Introduction to MongoDB

MongoDB is a widely-used, open-source, NoSQL database management system known for its flexibility, scalability, and high performance. It falls under the category of document-oriented databases, and its primary data model is the BSON (Binary JSON) format. MongoDB is designed to handle large volumes of data, making it suitable for a wide range of applications, from small projects to enterprise-level systems.

1.3.8.2 Example MongoDB Code and Explanation

```
const { MongoClient } = require('mongodb');

async function run() {

    const uri = 'mongodb://localhost:27017'; // Connection URI

    const client = new MongoClient(uri, { useUnifiedTopology: true }

);

try {

    await client.connect(); // Connect to the MongoDB server

    const database = client.db('mydb'); // Access a specific database

    const collection = database.collection('mycollection');

    // Access a specific collection

    const document = {

    name: 'John Doe', age: 30, email: 'johndoe@example.com' };

    const result = await collection.insertOne(document); // Insert a document
    console.log(`Inserted document with ID: ${result.insertedId}`);

    } finally {

        await client.close(); // Close the MongoDB client

    }
```

```
}

run().catch(console.error);
```

1. We begin by importing the MongoClient class from the MongoDB Node.js driver.
2. We create an async function named run to encapsulate the database operations.
3. We define the connection URI, which specifies the server location (localhost) and port (27017) for the MongoDB server. We also specify the useUnifiedTopology option for a more modern connection setup.
4. Inside the try block, we use the await client.connect() method to establish a connection to the MongoDB server.
5. We access a specific database named 'mydb' and a collection named 'mycollection' within that database.
6. We create a JavaScript object, document, to represent the data we want to insert into the collection.
7. Using collection.insertOne(document), we insert the document into the collection.
8. Finally, we close the MongoDB client with await client.close() to release resources.

1.4 ABOUT ORGANISATION

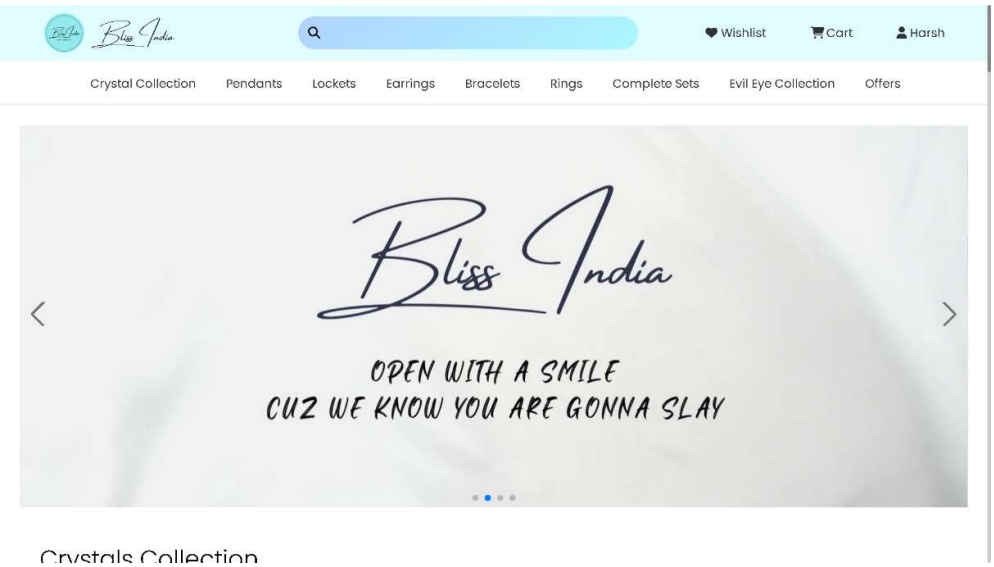
COLORATION MEDICARE LLP stands as a dynamic and versatile company, originally making its mark in the realm of cosmetics and more recently expanding its horizons into the jewelry business. Founded by the visionary entrepreneur Pramod Kr Jha, the company has evolved into a prominent player in the beauty and fashion industry. With a foundation rooted in cosmetics, COLORATION MEDICARE LLP has demonstrated adaptability and innovation by venturing into the intricate and expressive world of jewelry.

Under the leadership of Pramod Kr Jha, the company has embraced a commitment to quality, aesthetics, and customer satisfaction. His visionary approach has not only steered the company towards diversification but has also paved the way for a seamless blend of beauty and adornment. The foray into the jewelry business reflects a strategic move to cater to the evolving tastes and preferences of a discerning clientele.

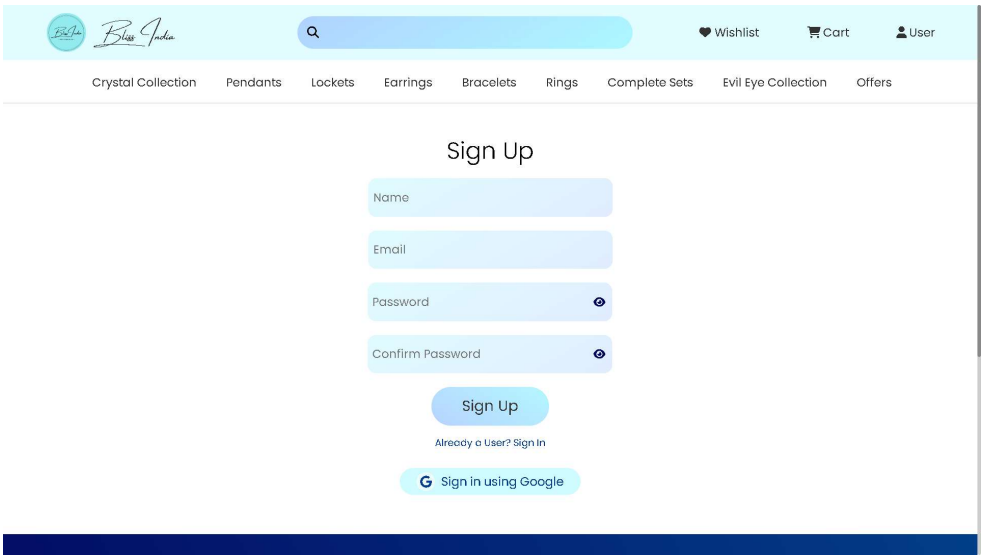
COLORATION MEDICARE LLP, guided by Pramod Kr Jha's vision, continues to carve its niche in the industry, offering a captivating fusion of cosmetics and jewelry. The brand embodies a commitment to excellence, innovation, and a customer-centric ethos, marking its presence as a notable and evolving entity in the competitive landscape of beauty and fashion.

2.PROJECT DESIGN

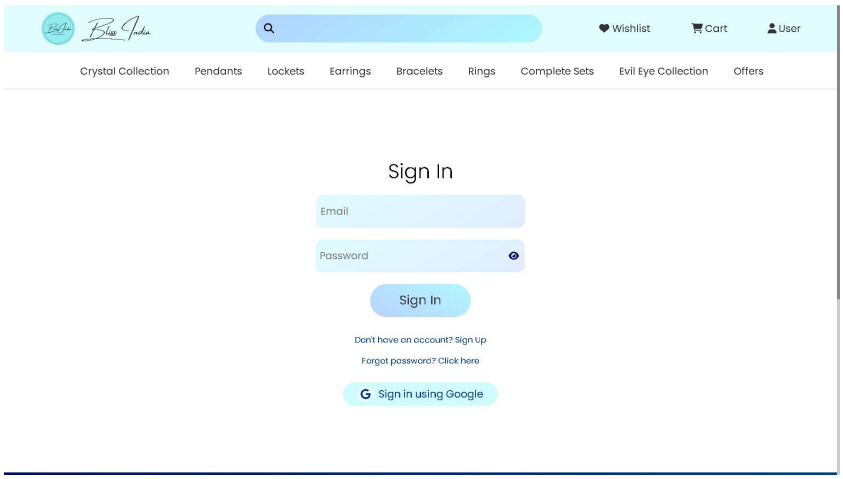
2.1 Client Side



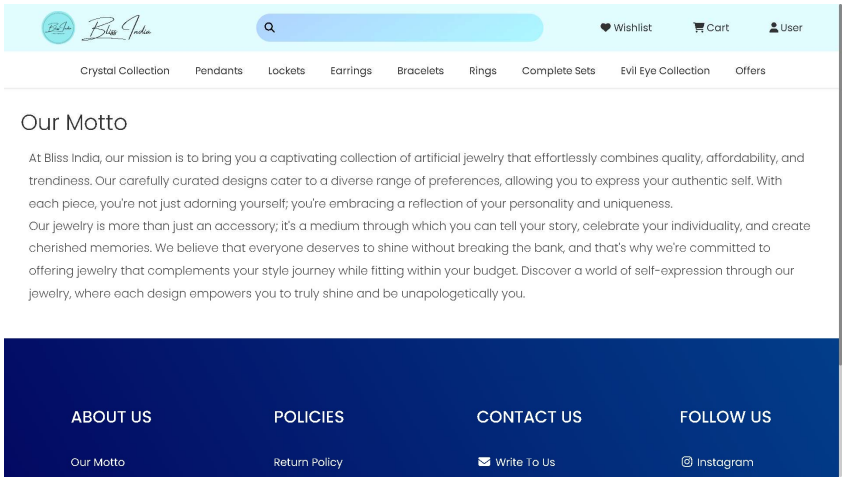
Home Page



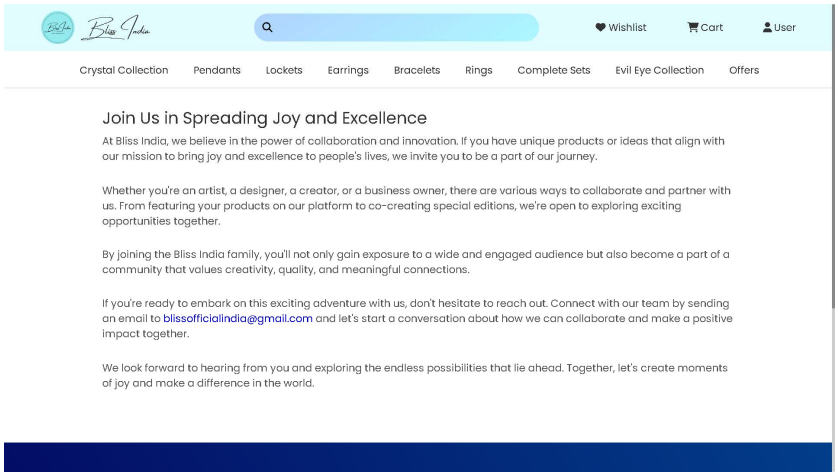
Signup Page



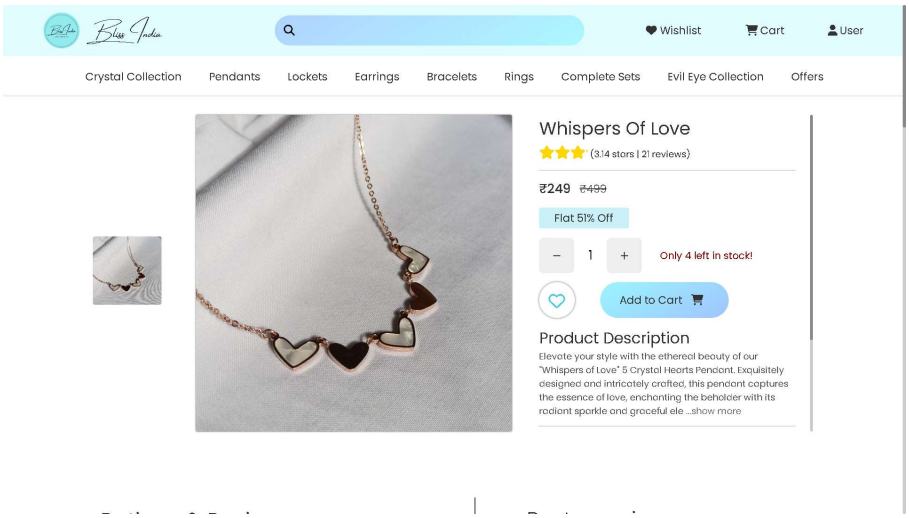
Signin Page



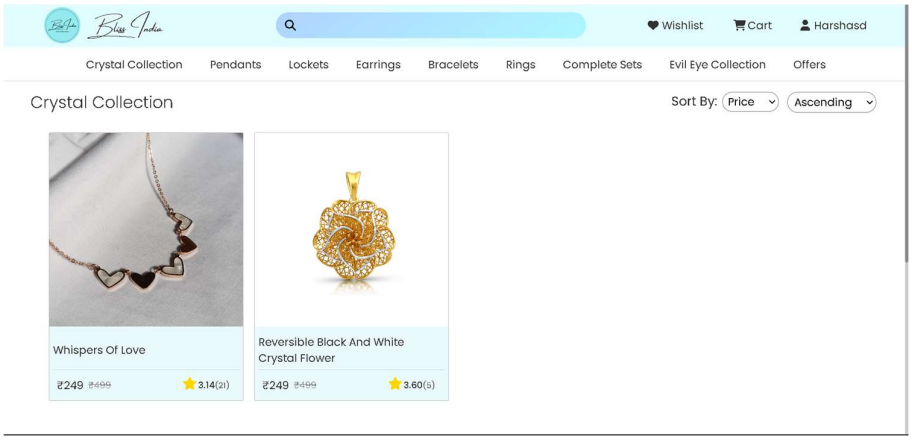
Our Motto Page



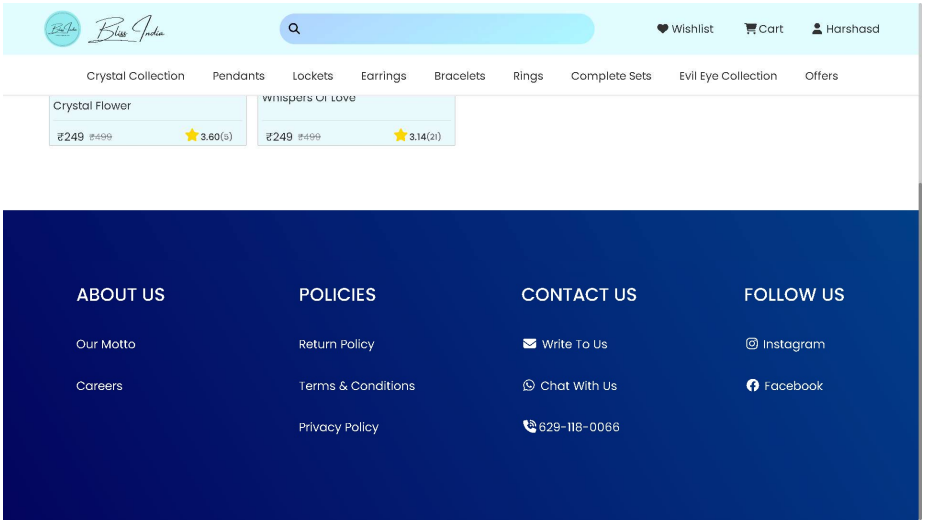
Careers Page



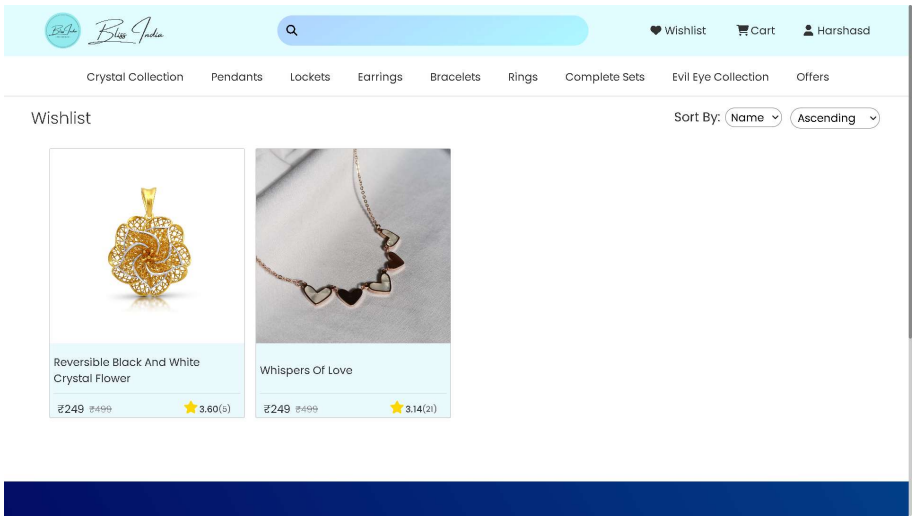
Products Page



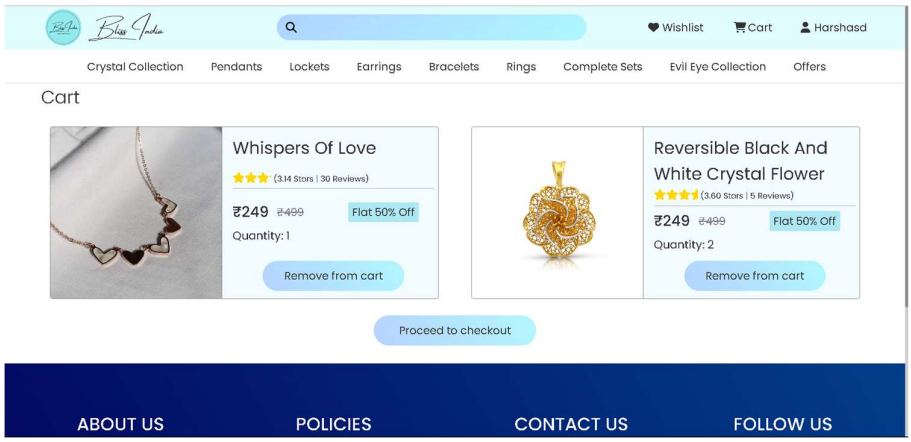
Our Jewellery Section



Footer Section

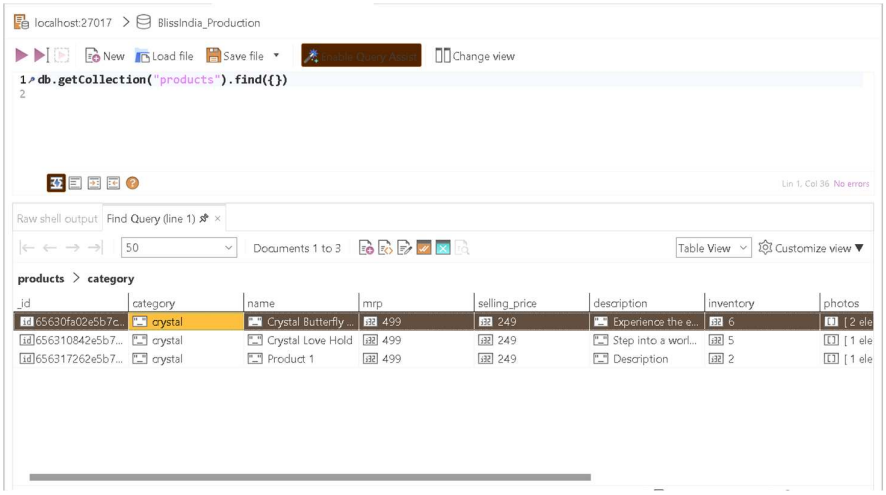


Wishlist Component

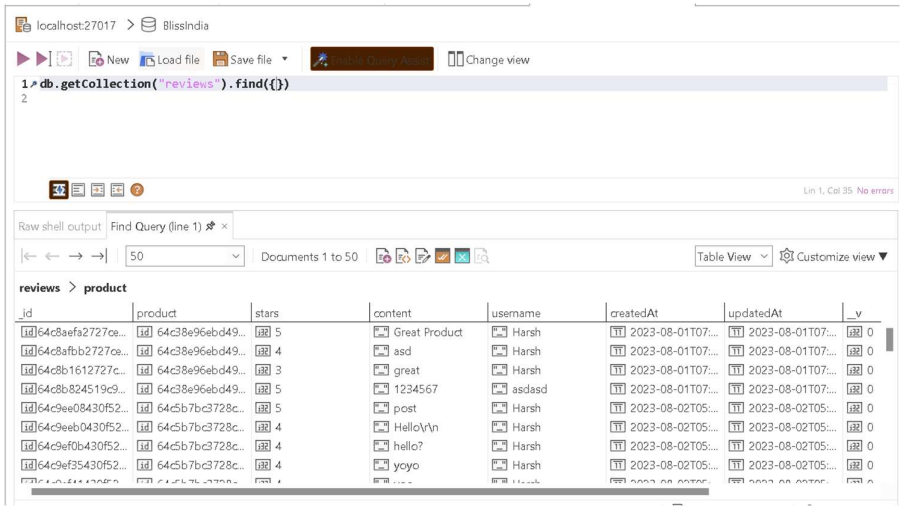


Cart Component

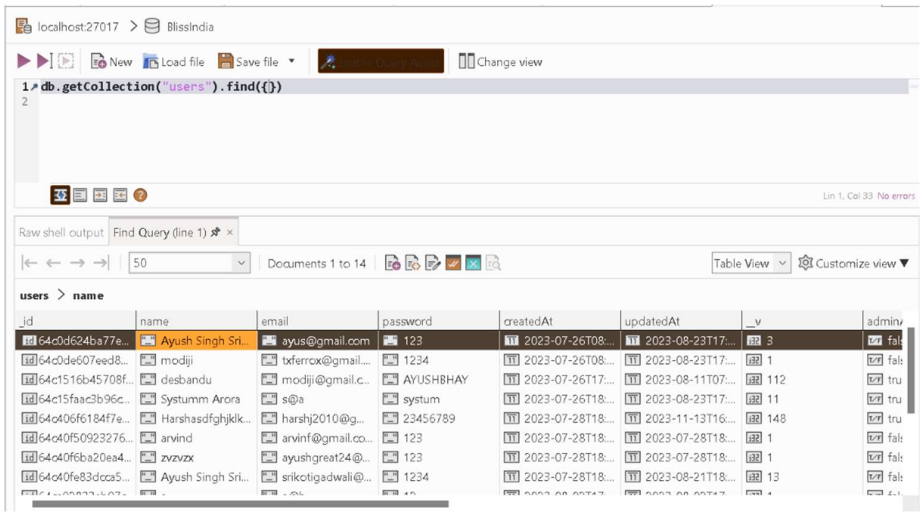
2.2 Server Side



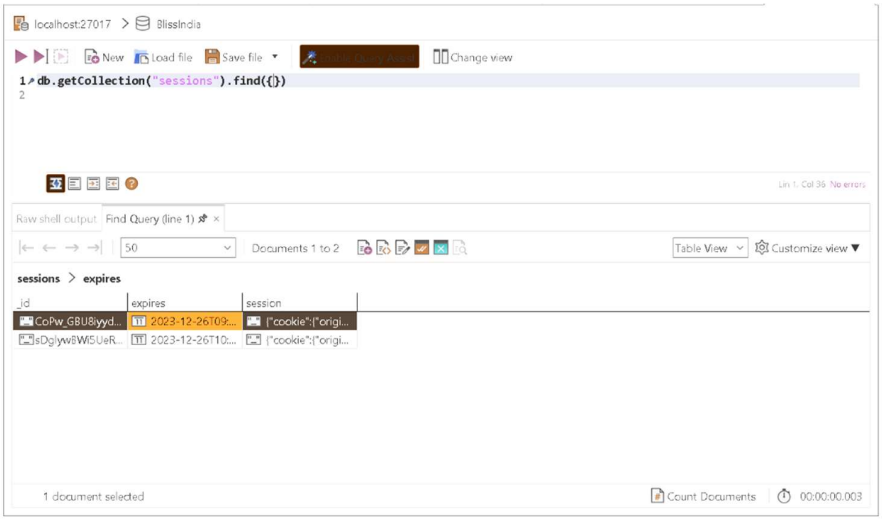
Product Collection



Review Collection



Users Collection



Session Collection

3. IMPLEMENTATION

3.1 Client Side Implementation

Client side also the front-end or fancy looking part of the website is styles using plain CSS. Let uss see how each component is implemented and designed using CSS one by one:

3.1.1. Navbar Component

3.1.1.1 Design and Implementation Code

```
<link rel="stylesheet" href="<%=assetPath('header.css')%>">

<header>

  <div class="company-logo">

    <a href="/"><img src="/Images/Bliss India Logo2.jpeg">Bliss India</a>

  </div>

  <div class="search-bar">

    <i class="fa-solid fa-magnifying-glass"></i>

    <form action="/products/search-product" method="post" id="search-form">

      <input type="text" required class="search-input" name="name" autocomplete="off">

    </form>

  </div>

  <div class="user-options">

    <div class="wishlist">

      <%if(locals.user){%>

        <!-- <div class="wish-count"></div> -->

        <a href="/products/wishlist?sort_by=name&order=1&user_id=<%=locals.user._id%>" id="wishlist-anchor"><i class="fa-solid fa-heart"></i>wishlist</a>

      <%}else{%>

        <a href="/products/wishlist?sort_by=name&order=1" id="wishlist-anchor"><i class="fa-solid fa-heart"></i>wishlist</a>

      <%}%>

    </div>

    <div class="cart">
```

```
        <a href="/products/cart" id="cart-anchor"><i class="fa-solid fa-cart-shopping"></i>Cart</a>
        <!-- <%if(locals.user && locals.user.cart.length>0){%>
            <div class="cart-count"></div>
        <%}%> -->
    </div>
    <%if(locals.user){%>
        <div class="user"><i class="fa-solid fa-user"></i>
            <%let name=user.name.split(' ')[0]%>
            <%let len=name.length%>
            <%=name.substring(0, Math.min(len, 8));%>
        </div>
    <%}else{%>
        <div class="user"><i class="fa-solid fa-user"></i> User</div>
    <%}%>
  </div>
</header>
```

3.1.1.2 Explanation

1. Stylesheet Inclusion: External stylesheet 'header.css' is linked using <link> with a dynamic asset path.
2. Header Section: Company logo (Bliss India) links to the home page. Search bar with an icon for user input. User options include Wishlist, Cart, and User Profile.
3. Navigation Bar: Categories like Crystal Collection, Pendants, Lockets, etc., are presented as clickable links.
4. User Hover Section: Dynamically displays links related to user actions, adjusting based on user login status.
5. Search Result Container: Empty <ul> for search results, populated dynamically based on user search input.
6. Font Awesome Icons: Font Awesome icon library is included, allowing the use of icons like a magnifying glass and user-related symbols.
7. JavaScript File Inclusion: 'header.js' script is included, indicating additional functionality managed by this script.

8. Search Functionality: Handles user input in the search bar. Makes AJAX requests to '/products/search-product'. Dynamically updates the search result container based on received data.

9. Search Result DOM Construction: searchResultDom function constructs HTML elements for each product search result.

10. Event Listeners: Listens for keyup events in the search bar. Triggers search functionality and dynamically updates the search result container.

3.1.2. Home.ejs

3.1.2.1 Implementation Code

```
<link rel="stylesheet" href="<%=assetPath('productCollection.css')%>">

<link rel="stylesheet" href="<%=assetPath('home.css')%>">

  <!-- <p class="tagline">Open with a smile</p>

  <p class="tagline">Cuz we know you are gonna SLAY</p> -->

<section class="banners">

  <div class="swiper banner">

    <div class="swiper-wrapper">

      <div class="swiper-slide image"><img src="/Images/Swiper/2.jpg" alt="Poster"></div>

      <div class="swiper-slide image"><video autoplay muted loop><source src="/Images/Swiper/5.mp4" type="video/mp4"></video></div>

      <!-- <div class="swiper-slide image"><img src="/Images/Swiper/1.jpg" alt=""></div> -->

      <div class="swiper-slide image"><img src="/Images/Swiper/4.jpg" alt="Poster"></div>

      <div class="swiper-slide image"><img src="/Images/Swiper/3.jpg" alt="Poster"></div>

    </div>

    <div class="swiper-button-next"></div>

    <div class="swiper-button-prev"></div>

    <div class="swiper-pagination"></div>

  </div>

</section>


  <main id="home-main">
```

```
<%for(let j of collection){%>
    <h2>
        <%if(j._id=='crystal'){%>
            <%=j._id%>s Collection
        <%}else if(j._id=='complete'){%>
            <%=j._id%> Sets
        <%}else if(j._id=='evil_eye'){%>
            Evil Eyes Collection
        <%}else {%>
            <%=j._id%>s
        <%}%>
    </h2>
    <div class="product-area">
        <%for(let i of j.products.splice(0,7)){%>
            <section class="individual-product">
                <a id="image-link" target="_blank" href="/products?id=<%=i._id%>">
                    <img src="<%=i.photos[0]%>">
                </a>
                <div class="product-details">
                    <p class="product-name"><a target="_blank" href="/products?id=<%=i._id%>">
                        <%=i.name%>
                    </a></p>
                    <div class="product-pricing">
                        <div>
                            <p class="selling-price"><i class="fa-solid fa-indian-rupee-sign"></i>
                                <%=i.selling_price%>
                            </p>
                            <p class="mrp" id="mrp"><i class="fa-solid fa-indian-rupee-sign"></i>
```

```
                <%=i.mrp%>
            </p>
        </div>
            <p id="stars">
                <i id="stars-star" class="fa-solid fa-star"></i>
<%=parseFloat(i.review_rating).toFixed(2)%><span>(<%=i.review.length%>)</span
>
            </p>
        </div>
    </div>
</section>
<%}%>
<section class="individual-product explore">


    <%if(j._id=='crystal'){%>
        <a href="/products/crystal-collection?sort_by=name&order=1"
class="explore-anchor">
    <%}else if(j._id=='complete'){%>
        <a href="/products/complete-sets?sort_by=name&order=1"
class="explore-anchor">
    <%}else if(j._id=='evil_eye'){%>
        <a href="/products/evil-eye-collection?sort_by=name&order=1"
class="explore-anchor">
    <%}else {%>
        <a href="/products/<%=j._id%>s?sort_by=name&order=1"
class="explore-anchor">
    <%}%>
        <div class="circle"><i class="fa-solid fa-arrow-right"></i></div>
        <p class="explore-text">Explore All</p>
    </a>
</section>
</div>
```

```
    <%}%>
  </main>
<script src="https://cdn.jsdelivr.net/npm/swiper@10/swiper-bundle.min.js"></script>
<script type="text/javascript" src="/js/home.js"></script>
```

3.1.2.2 Explanation

1. Main Section (home-main): Iterates through collections, displaying dynamic headings and up to 7 products per category.
2. Banner Section: Utilizes SwiperJS to create a banner section with image and video slides. Navigation buttons, pagination, and autoplay enhance the visual experience.
3. Dynamic Headings: Conditions generate headings like "Crystal Collection," "Complete Sets," and "Evil Eyes Collection."
4. Explore Section: "Explore All" link for each category redirects users with a circular arrow icon.
5. Review Stars: Displays product review ratings using stars, showing the average rating and the number of reviews.
6. Script Inclusion: Imports SwiperJS library and 'home.js' script for added functionality.

3.2 Server Side Implementation

Server-side designing is done through MongoDB for database as it is very efficient for NoSQL data structures as in my case and for server, I used Node.js and Express.js for API and Server-side rendering. Let us look through at some snippets and the explanation behind their implementation:

3.2.1 Server

Using Node.js and Express.js

a) Code Implementation

```
require("dotenv").config();
const express = require("express");
sconst mongoose = require("mongoose");
const cors = require("cors");
const productRoutes = require("./routes/productRoutes");
const adminRoutes = require("./routes/adminRoutes");
const customerRoutes = require("./routes/customerRoutes");
```

```
const app = express();
app.use(express.json({ limit: "50mb" }));
app.use(cors());
app.use("/api/products", productRoutes);
app.use("/api/admin", adminRoutes);
app.use("/api/customer", customerRoutes);

mongoose
.connect('mongodb://127.0.0.1:27017/BlissIndia')
.then(() => { app.listen(3000, () => {
console.log("database connected & port is running!", 3000);
});
})
.catch((error) => { console.log(error);
});
```

b) Explanation

1. Configuration: It configures the server by loading environment variables from a .env file using dotenv. It also sets up necessary middleware, including express.json to parse JSON data and cors for enabling Cross-Origin Resource Sharing.
2. Routing: The code defines three main routes for different parts of the application: product routes, admin routes, and customer routes. These routes are imported from separate route files.
3. Database Connection: It connects to a MongoDB database hosted locally on mongodb://127.0.0.1:27017/BlissIndia.
4. Server Initialization: Once the database connection is established, the server starts listening on port 3000. If the connection is successful, it logs "database connected & port is running!" to the console, indicating that the server is up and running.

3.2.2 Database

Design Using Mongoose and MongoDB

a) Code Implementation

```
const mongoose = require("mongoose");
const Schema = mongoose.Schema;
const productSchema = new Schema({
```

```
    category: { type: String, required: true,
},
name: {
    type: String,
    required: [true, "Please add a product Name"],
},
price: {
    type: Number, required: true,
},
imgOne: {
    public_id: { type: String, required: true,
},
url: {
    type: String,….

module.exports = mongoose.model("Product", productSchema);
```

b) Explanation

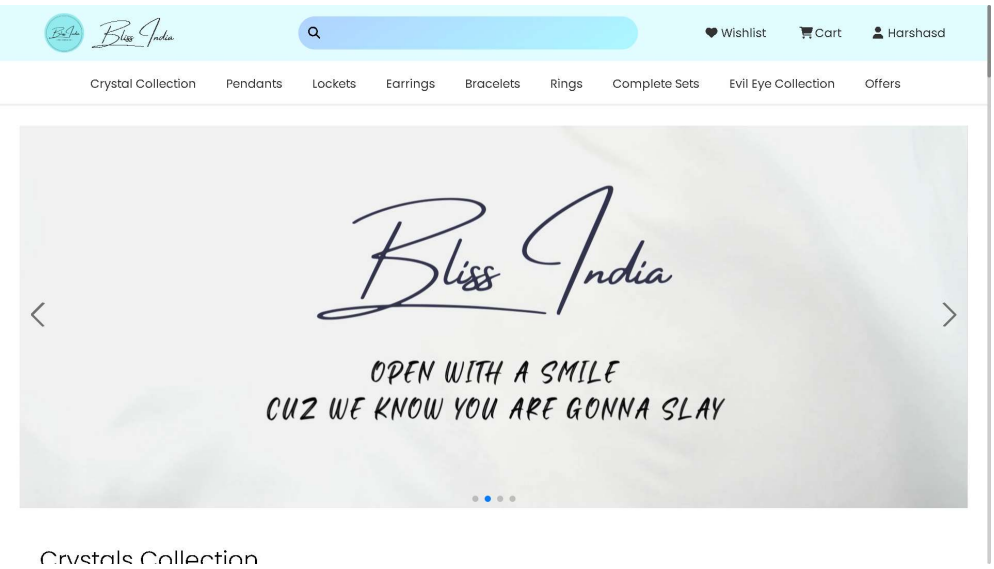
1. Mongoose and Schema Import: The code first imports the necessary modules, Mongoose and Schema, for defining a schema.
2. Product Schema: The productSchema is defined using the new Schema constructor. It specifies the structure and properties of the product model, including:
3. category: A string field representing the category of the product. It's marked as required.
4. name: A string field representing the name of the product. It's required and includes a custom error message for missing data.
5. price: A number field representing the price of the product. It's marked as required.
6. imgOne: An object with two fields, public_id and url, representing an image of the product. Both fields are required.
7. imgTwo: Similar to imgOne, but with an optional public_id field.
8. description: An array of objects, where each object represents details of the product. Each object has two fields, detailOne and detailTwo, both required.

9. Timestamps: The schema is configured to include timestamps with { timestamps: true }. This means that each document will automatically include createdAt and updatedAt timestamps.
10. Model Export: The mongoose.model method is used to create a Mongoose model named "Product" based on the defined schema. This model can be used to interact with the MongoDB database for CRUD (Create, Read, Update, Delete) operations on product data.
11. Comments: The code includes commented-out sections that seem to represent images (imgOne and imgTwo). These sections might have been removed or commented for specific reasons, and they could be related to image upload functionality for products.

4. RESULT AND DISCUSSION

4.1 Result

The adept navigation and resolution of the myriad challenges encountered during the developmental phases culminated in the triumphant deployment of BlissIndia. This e-commerce platform stands not only as a testament to the project's success but also as a manifestation of a functional and user-friendly interface, meticulously crafted to offer patrons a superlative and seamless shopping expedition. The fruition of the project resonates with the unwavering dedication and unparalleled problem-solving prowess exhibited by the adept development team. Their concerted efforts and adept maneuvering through complexities have not only birthed a digital retail space but have sculpted an immersive and gratifying shopping environment for customers, wherein each click and interaction resonates with the culmination of strategic decision-making and innovative solutions. BlissIndia's successful realization is, undeniably, a product of tenacity, ingenuity, and a collective commitment to surmounting challenges in the pursuit of delivering excellence.



Result of the Project

4.2 Discussion

While developing BlissIndia, we encountered several challenges that required immediate problem-solving and some-guidance. Challenges that I encountered were:

Challenge 1: Technical Complexity

The project involved various technologies such as MongoDB, CSS, JS and third-party integrations such as SwiperJS library, MulterJS, Google Auth which posed a high-end learning curve for the time period.

Challenge 2: Data Management

Efficiently managing and organizing the vast amount of product data, customer information, and order details proved to be a complex task. But since, I knew the database management with MongoDB beforehand, it was not quite a challenge to me.

Challenge 3: User Experience (UX) Design

Ensuring an intuitive and aesthetically pleasing user interface that met the expectations of modern e-commerce websites was a design challenge, this design part was the most time-consuming part of the whole project.

Challenge 4: Security and Privacy

Safeguarding user data and ensuring secure online transactions where critical challenges was not much of a task, but the bigger challenge was to implement this in the code.

5. FUTURE SCOPE & CONCLUSION

5.1 Future Scope

The development of BlissIndia, a full-stack e-commerce website, marks a significant achievement in terms of both learning and practical application of web development technologies. While the project has reached its current stage of completion, it is essential to recognize the numerous opportunities for future enhancement and expansion. The following are some of the key areas for future development:

5.1.1 Enhanced User Experience:

The future of BlissIndia lies in continually improving the user experience. This includes further refinements in the website's design and navigation to create a more immersive and intuitive shopping environment. Employing user feedback and implementing advanced UX design principles will be pivotal in achieving this goal.

5.1.2 Expansion of Product Catalog:

BlissIndia can diversify its product catalog by introducing a wider range of jewelry and accessories, staying current with the latest trends and emerging fashion. This expansion can cater to a broader customer base and offer more choices to consumers.

5.1.3 Geographical Expansion:

Expanding the reach of BlissIndia to serve customers in different geographic regions can be a significant step. This could involve partnerships with local distributors, multi-language support, and optimized logistics to ensure a seamless global shopping experience.

5.1.4 Marketing and SEO:

Implementing a robust digital marketing strategy and search engine optimization (SEO) plan can enhance BlissIndia's online visibility. This can involve content
marketing, social media engagement, and paid advertising to attract more traffic and potential customers.

5.1.5 Personalized User Recommendations:

Introducing a personalized recommendation system based on user preferences and past purchase history can elevate the shopping experience. Utilizing machine learning algorithms, BlissIndia can offer tailored product suggestions, enhancing customer engagement and satisfaction.

5.1.6 Mobile App Development:

Creating a dedicated mobile application can open new avenues for customer interaction. A mobile app provides convenience and accessibility, allowing users to browse, shop, and receive updates on the go. This move can significantly contribute to brand loyalty.

5.1.7 Virtual Try-On Features:

Implementing virtual try-on features for jewelry items can bridge the gap between online and offline shopping experiences. Utilizing augmented reality (AR) technology, customers can virtually try on jewelry before making a purchase, boosting confidence in their buying decisions.

5.1.8 Community Engagement and Events:

Fostering a sense of community among BlissIndia customers through online forums, social media groups, or exclusive events can deepen brand loyalty. Engaging users in discussions, soliciting feedback, and organizing virtual or physical events can create a vibrant brand community.

5.2 Conclusion

In conclusion, the journey of creating BlissIndia has been an invaluable experience that has not only honed technical skills but also instilled a deep appreciation for the challenges and opportunities within the realm of e-commerce and web development. BlissIndia, the full-stack e-commerce website, is more than just a learning exercise; it is a tangible manifestation of the possibilities that emerge when technology, creativity, and determination converge. The website provides a robust platform for tech enthusiasts and shoppers alike, offering a diverse array of jewelry, accessories, and a seamless shopping experience.

As we look toward the future, the scope for BlissIndia's growth and evolution is promising. The continual pursuit of excellence in user experience, mobile integration, product expansion, geographical reach, and effective marketing strategies will ensure that BlissIndia remains a thriving destination for fashion-trendy consumers.

The development of BlissIndia and the experiences gained throughout this project serve as a stepping stone for future endeavors in the ever-evolving landscape of web development and e-commerce. It is a testament to the potential for innovation and progress in this dynamic field.

With a steadfast commitment to quality and a vision for the future, BlissIndia is poised to grow and evolve, continuously meeting the needs of its customers and embracing the

exciting opportunities that lie ahead. The journey is ongoing, and the destination is limitless. BlissIndia is not just a project; it is my my step to learn via development.

Full-stack web development is a dynamic and ever-evolving field, and this journey is far from its end; it is, in fact, the start of a continuous quest for excellence and innovation. My experiences throughout this training have laid the foundation for a promising and rewarding future in the world of web development, where the horizons are limitless, and the opportunities are boundless.

REFERENCES

1. HTML, CSS(W3schools): https://www.w3schools.com/html/html_css.asp
2. JavaScript(MDN Docs): https://developer.mozilla.org/en-US/docs/Web/JavaScript
3. JavaScript Advanced: https://www.youtube.com/watch?v=SBmSRK3feww
4. Node.js(Official Documentation): https://nodejs.org/en/docs
5. Express.js(MDN Docs): https://developer.mozilla.org/en-US/docs/express/
6. MongoDB(Official Documentation): https://www.mongodb.com/docs/
7. Historical Reference (Wikipedia): https://www.wikipedia.org/
8. SwiperJS (Official Documentation): https://swiperjs.com/
9. NotyJS (Official Documentation): https://www.npmjs.com/package/noty